SQL Code

```sql
CREATE DATABASE retail_store_mp;
USE retail_store_mp;

CREATE TABLE customers (
                customer_id INT PRIMARY KEY AUTO_INCREMENT,name VARCHAR(100) NOT NULL,
                email VARCHAR(100) NOT NULL UNIQUE,
                phone VARCHAR(15),
                created_at DATETIME DEFAULT CURRENT_TIMESTAMP
                );

CREATE TABLE products (
                product_id INT PRIMARY KEY AUTO_INCREMENT,
                name VARCHAR(100) NOT NULL,
                category VARCHAR(50) NOT NULL,
                price DECIMAL(10,2) NOT NULL,
                stock_quantity INT NOT NULL DEFAULT 0,
                added_on DATETIME DEFAULT CURRENT_TIMESTAMP
        );

CREATE TABLE orders (
                order_id INT PRIMARY KEY AUTO_INCREMENT,
                customer_id INT,
                order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
        status VARCHAR(20) DEFAULT 'Pending',
                total_amount DECIMAL(10,2),
                FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
        );

CREATE TABLE order_items (
                order_item_id INT PRIMARY KEY AUTO_INCREMENT,
                order_id INT,
                product_id INT,
                quantity INT NOT NULL CHECK (quantity > 0),
                item_price DECIMAL(10,2) NOT NULL,
                FOREIGN KEY (order_id) REFERENCES orders(order_id),
                FOREIGN KEY (product_id) REFERENCES products(product_id)
        );

CREATE TABLE payments (
                payment_id INT PRIMARY KEY AUTO_INCREMENT,
                order_id INT,
                payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
                amount_paid DECIMAL(10,2) NOT NULL CHECK (amount_paid > 0),
                method VARCHAR(20) NOT NULL,
                FOREIGN KEY (order_id) REFERENCES orders(order_id)
        );

CREATE TABLE product_reviews (
                review_id INT PRIMARY KEY AUTO_INCREMENT,
                product_id INT,
```

```
                customer_id INT,
                rating INT NOT NULL,
                review_text TEXT,
                review_date DATETIME DEFAULT CURRENT_TIMESTAMP,
                FOREIGN KEY (product_id) REFERENCES products(product_id),
                FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
        );
SELECT *
FROM CUSTOMERS;

# LEVEL 01---BASIC--------->
# 1. Retrieve customer names and emails for email marketing--->

SELECT NAME,EMAIL
FROM CUSTOMERS;

# 2. View complete product catalog with all available details--->

SELECT *
FROM PRODUCTS;

# 3. List all unique product categories--->

SELECT DISTINCT CATEGORY
FROM PRODUCTS;

# 4. Show all products priced above ₹1,000--->

SELECT *
FROM PRODUCTS
WHERE PRICE > 1000;

# 5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)--->

SELECT *
FROM PRODUCTS
WHERE PRICE BETWEEN 2000 AND 5000;

# 6. Fetch data for specific customer IDs (e.g., from loyalty program list)--->

WITH CUSTOMER_ORDER_COUNTS AS (
    SELECT
        CUSTOMER_ID,
        COUNT(ORDER_ID) AS TOTAL_ORDERS
    FROM ORDERS
    GROUP BY CUSTOMER_ID
),
LOYALTY_CATEGORIZATION AS (
    SELECT
        CUSTOMER_ID,
        TOTAL_ORDERS,
        CASE
            WHEN TOTAL_ORDERS >= 15 THEN 'LOYAL'
            WHEN TOTAL_ORDERS >= 7 THEN 'POTENTIAL LOYAL'
```

```
            WHEN TOTAL_ORDERS >= 3 THEN 'HIGH SPENDER'
            ELSE 'RECENT JOINEES'
        END AS LOYALTY_PROGRAM
    FROM CUSTOMER_ORDER_COUNTS
)
SELECT
    CUSTOMER_ID,
    TOTAL_ORDERS,
    LOYALTY_PROGRAM
FROM LOYALTY_CATEGORIZATION;

# 7. Identify customers whose names start with the letter ‘A’--->

SELECT NAME , CUSTOMER_ID
FROM CUSTOMERS
WHERE NAME LIKE "A%";

# 8. List electronics products priced under ₹3,000--->

SELECT *
FROM PRODUCTS
WHERE CATEGORY = "ELECTRONICS" AND PRICE < 3000;

# 9. Display product names and prices in descending order of price--->

SELECT NAME , PRICE
FROM PRODUCTS
ORDER BY PRICE DESC;

# 10. Display product names and prices, sorted by price and then by name--->

SELECT NAME , PRICE
FROM PRODUCTS
ORDER BY PRICE ASC , NAME ASC;

# LEVEL 02---FILTERRING AND FORMATTING--------->
# 1. Retrieve orders where customer information is missing (possibly due to data
migration or deletion)--->

SELECT
        O.ORDER_ID,
        O.ORDER_DATE,
        O.TOTAL_AMOUNT
FROM
    ORDERS O
JOIN
    CUSTOMERS C
ON
    O.CUSTOMER_ID = C.CUSTOMER_ID
WHERE
    O.CUSTOMER_ID IS NULL
    OR TRIM(O.CUSTOMER_ID) = ''
    OR C.CUSTOMER_ID IS NULL;

```

```sql
# 2. Display customer names and emails using column aliases for frontend readability-->

SELECT
        NAME AS CUSTOMER_NAME,
        EMAIL AS CUSTOMER_EMAIL
FROM
    CUSTOMERS;

# 3. Calculate total value per item ordered by multiplying quantity and item price--->

SELECT
        ORDER_ITEM_ID,
        SUM(QUANTITY * ITEM_PRICE) AS TOTAL_VALUE
FROM
    ORDER_ITEMS
GROUP BY
    ORDER_ITEM_ID;

# 4. Combine customer name and phone number in a single column--->

SELECT
    CONCAT(NAME , "---->" , PHONE) AS CUSTOMER_DETAILS
FROM
    CUSTOMERS;

# 5. Extract only the date part from order timestamps for date-wise reporting--->

SELECT
        DATE(ORDER_DATE) AS CUSTOMER_ORDER_DATE,
        COUNT(*) AS TOTAL_ORDER
FROM
    ORDERS
GROUP BY
    DATE(ORDER_DATE)
ORDER BY
    CUSTOMER_ORDER_DATE DESC;

# 6. List products that do not have any stock left--->

SELECT
        PRODUCT_ID,
        NAME,
        STOCK_QUANTITY
FROM
    PRODUCTS
WHERE
    STOCK_QUANTITY = 0;

# LEVEL 03---AGGREGATION--------->
# 1. Count the total number of orders placed--->

SELECT COUNT(*) AS TOTAL_ORDER
FROM ORDERS;
```

```sql

# 2. Calculate the total revenue collected from all orders--->

SELECT SUM(AMOUNT_PAID) AS TOTAL_REVENUE
FROM PAYMENTS;

# 3. Calculate the average order value--->

SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
FROM ORDERS;

# 4. Count the number of customers who have placed at least one order--->

SELECT
        COUNT(DISTINCT CUSTOMER_ID) AS NUMBER_OF_CUSTOMER
FROM
    ORDERS;

# 5. Find the number of orders placed by each customer--->

SELECT
        COUNT(DISTINCT ORDER_ID) AS NUMBER_OF_ORDERS , CUSTOMER_ID
FROM
    ORDERS
GROUP BY
    CUSTOMER_ID;

# 6. Find total sales amount made by each customer--->

SELECT
    DISTINCT SUM(TOTAL_AMOUNT) AS SALES_AMOUNT_PER_CUSTOMER , CUSTOMER_ID
FROM
    ORDERS
GROUP BY
    CUSTOMER_ID;

# 7. List the number of products sold per category--->

SELECT
        COUNT(DISTINCT PRODUCT_ID) , CATEGORY
FROM
    PRODUCTS
GROUP BY
    CATEGORY;

# 8. Find the average item price per category--->

SELECT
        ROUND(AVG(PRICE),2) , CATEGORY
FROM
    PRODUCTS
GROUP BY
    CATEGORY;

```

```sql
# 9. Show number of orders placed per day--->

SELECT
        COUNT(DISTINCT ORDER_ID) AS NUMBER_OF_ORDER , ORDER_DATE
FROM
    ORDERS
GROUP BY
    ORDER_DATE
ORDER BY
    ORDER_DATE DESC;

# 10. List total payments received per payment method--->

SELECT
    SUM(DISTINCT AMOUNT_PAID) AS TOTAL_PAYMENTS , METHOD
FROM
    PAYMENTS
GROUP BY
    METHOD;

# LEVEL 04---MULTI-TABLE QUERRIES (JOINS)--------->
# 1. Retrieve order details along with the customer name (INNER JOIN)--->

SELECT
    O.ORDER_ID,
    O.ORDER_DATE,
    O.TOTAL_AMOUNT,
    O.STATUS,
    C.NAME
FROM
    ORDERS O
JOIN
    CUSTOMERS C
ON
    O.CUSTOMER_ID = C.CUSTOMER_ID;

# 2. Get list of products that have been sold (INNER JOIN with order_items)--->

SELECT
    DISTINCT P.PRODUCT_ID,
    P.NAME
FROM
    PRODUCTS P
JOIN
    ORDER_ITEMS OI
ON
    P.PRODUCT_ID = OI.PRODUCT_ID;

# 3. List all orders with their payment method (INNER JOIN)--->

SELECT
         DISTINCT O.ORDER_ID,
     P.METHOD
FROM
```

```
      ORDERS O
JOIN
      PAYMENTS P
ON
      O.ORDER_ID = P.ORDER_ID;

# 4. Get list of customers and their orders (LEFT JOIN)--->

SELECT
      C.CUSTOMER_ID,
      C.NAME,
      O.ORDER_ID,
      O.ORDER_DATE
FROM
      CUSTOMERS C
LEFT JOIN
      ORDERS O
ON
      C.CUSTOMER_ID = O.CUSTOMER_ID;

# 5. List all products along with order item quantity (LEFT JOIN)--->

SELECT
      DISTINCT P.PRODUCT_ID,
      P.NAME,
      OII.QUANTITY
FROM
      PRODUCTS P
LEFT JOIN
      ORDER_ITEMS OII
ON
      P.PRODUCT_ID = OII.PRODUCT_ID;

# 6. List all payments including those with no matching orders (RIGHT JOIN)--->

SELECT
      P.PAYMENT_ID,
      P.PAYMENT_DATE,
      P.AMOUNT_PAID,
      O.ORDER_ID,
      O.ORDER_DATE
FROM
      ORDERS O
RIGHT JOIN
      PAYMENTS P
ON
      O.ORDER_ID = P.ORDER_ID;

# 7. Combine data from three tables: customer, order, and payment--->

SELECT
      C.CUSTOMER_ID,
      C.NAME,
      O.ORDER_ID,
```

```sql
      O.ORDER_DATE,
      P.PAYMENT_ID,
      P.PAYMENT_DATE,
      P.AMOUNT_PAID
FROM
      CUSTOMERS C
JOIN
      ORDERS O
ON
      C.CUSTOMER_ID = O.CUSTOMER_ID
JOIN
      PAYMENTS P
ON
      O.ORDER_ID = P.ORDER_ID;

# LEVEL 05---SUBQUERIES(INNER QUERIES)--------->
# 1. List all products priced above the average product price--->

SELECT
      PRODUCT_ID , NAME , PRICE
FROM
      PRODUCTS
WHERE
      PRICE > (
                SELECT AVG(PRICE)
                FROM PRODUCTS
                )
ORDER BY PRICE DESC;

# 2. Find customers who have placed at least one order--->

SELECT
      CUSTOMER_ID , NAME
FROM
      CUSTOMERS
WHERE
      CUSTOMER_ID
IN (
      SELECT CUSTOMER_ID
      FROM ORDERS
        );

# 3. Show orders whose total amount is above the average for that customer--->

SELECT
      O1.ORDER_ID , O1.CUSTOMER_ID , O1.TOTAL_AMOUNT
FROM
      ORDERS O1
WHERE
      O1.TOTAL_AMOUNT > (
                         SELECT AVG(O2.TOTAL_AMOUNT)
                         FROM ORDERS O2
                         WHERE O2.CUSTOMER_ID = O1.CUSTOMER_ID
                         );
```

```sql

# 4. Display customers who haven't placed any orders--->

SELECT
      CUSTOMER_ID , NAME
FROM
      CUSTOMERS
WHERE
      CUSTOMER_ID
NOT IN (
      SELECT CUSTOMER_ID
      FROM ORDERS
        );

# 5. Show products that were never ordered--->

SELECT
      PRODUCT_ID , NAME
FROM
      PRODUCTS
WHERE
      PRODUCT_ID
NOT IN (
        SELECT PRODUCT_ID
                FROM ORDER_ITEMS
        WHERE PRODUCT_ID IS NOT NULL
        );

# 6. Show highest value order per customer--->

SELECT
      O.CUSTOMER_ID , O.ORDER_ID , O.TOTAL_AMOUNT
FROM
      ORDERS O
JOIN (
      SELECT CUSTOMER_ID , MAX(TOTAL_AMOUNT) AS MAX_ORDER_VALUE
      FROM ORDERS
      GROUP BY CUSTOMER_ID
      ) MO
ON O.CUSTOMER_ID = MO.CUSTOMER_ID
AND O.TOTAL_AMOUNT = MO.MAX_ORDER_VALUE;

# 7. Highest Order Per Customer (Including Names)--->

SELECT
      C.NAME,
      O.ORDER_ID,
      O.TOTAL_AMOUNT
FROM
      ORDERS O
JOIN (
      SELECT CUSTOMER_ID , MAX(TOTAL_AMOUNT) AS MAX_ORDERVALUE
      FROM ORDERS
      GROUP BY CUSTOMER_ID
```

```
      ) MO
ON    O.CUSTOMER_ID = MO.CUSTOMER_ID
AND   O.TOTAL_AMOUNT = MO.MAX_ORDERVALUE
JOIN  CUSTOMERS C
ON    O.CUSTOMER_ID = C.CUSTOMER_ID;

# LEVEL 06---SET OPERATIONS--------->
# 1. List all customers who have either placed an order or written a product review--->

SELECT
      CUSTOMER_ID , NAME , EMAIL
FROM
      CUSTOMERS
WHERE CUSTOMER_ID IN (
                      SELECT CUSTOMER_ID
                      FROM ORDERS
                      UNION
                      SELECT CUSTOMER_ID
                      FROM PRODUCT_REVIEWS
                      );

# 2. List all customers who have placed an order as well as reviewed a product [intersect not supported]--->

SELECT
      C.CUSTOMER_ID,
      C.NAME,
      C.EMAIL
FROM CUSTOMERS C
WHERE CUSTOMER_ID IN (
                                          SELECT CUSTOMER_ID
                      FROM (
                            SELECT DISTINCT CUSTOMER_ID
                            FROM ORDERS
                            UNION ALL
                            SELECT DISTINCT CUSTOMER_ID
                            FROM PRODUCT_REVIEWS
                            ) AS COMBINED
                      GROUP BY CUSTOMER_ID
                      HAVING COUNT(*) = 2
                      );
```